

Práctica 2

¿Cómo realizar la limpieza y análisis de los datos?

Metadatos

- **Integrantes del grupo:**
 - Agustín Barbatelli Balboa
 - Adrián Cordones Martínez
- **Enlace al repositorio con el código:**
<https://github.com/adriancordones/steam-apps-analysis>
- **Enlace al vídeo de explicación:**
<https://...>

Las librerías necesarias ...

```
library(dplyr)
library(stringr)
library(knitr)
library(httr)

knitr::opts_chunk$set(echo = TRUE)
readRenviron(".Renviron")
```

1 Descripción

Este dataset recoge metadatos de aplicaciones publicadas en la *Steam Store*, extraídos mediante web scraping de las páginas individuales de producto. Cada registro corresponde a un juego o aplicación e incluye información identificativa (título, desarrollador, editor), condiciones comerciales (precio, descuento), recepción por parte de los usuarios (número y resumen general de reseñas), características de contenido (DLCs, logros, idiomas, banda sonora) y clasificación temática (géneros y etiquetas de la comunidad). Los datos reflejan el estado de la tienda en el momento de la extracción (abril de 2026) y cubren los títulos más populares y recientes según los resultados de búsqueda de Steam para España y con la página en español.

2 Integración y selección

```
...

# Carga del dataset
df <- read.csv("../datasets/steam_apps.csv", sep = ",", dec = ".")
```

```
# TODO: Arreglar para que no se rompan los caracteres de TM, (R), etc.

# Función de ayuda
get_var <- function(data, path) {
  if (is.null(data)) return(NA)
  result <- data
  for (key in path) {
    result <- result[[key]]
    if (is.null(result)) return(NA)
  }
  return(result)
}

# Función genérica para requests
get_response <- function(url, cooldown = 10) {
  response <- tryCatch(GET(url, timeout(cooldown)), error = function(e) NULL)
  if (is.null(response) || status_code(response) != 200) return(NULL)
  data <- content(response, as = "parsed", type = "application/json")

  return(data)
}

# Función para consultas a API de Steam: restricciones de edad y reviews
get_steam <- function(app_id) {
  url <- paste0("https://store.steampowered.com/appreviews/", app_id, "?json=1&purchase_type=all&num_pe")
  data <- get_response(url)
  total_positive <- get_var(data, c("query_summary", "total_positive"))
  total_reviews <- get_var(data, c("query_summary", "total_reviews"))

  url <- paste0("https://store.steampowered.com/api/appdetails?appids=", app_id)
  data <- get_response(url)
  app_id <- as.character(app_id)
  required_age <- get_var(data, c(app_id, "data", "required_age"))
  pegi <- get_var(data, c(app_id, "data", "ratings", "pegi", "rating"))

  result <- list(total_positive = total_positive, total_reviews = total_reviews,
                required_age = required_age, pegi = pegi)

  return(result)
}

# Función para consultas a API de SteamSpy: ventas estimadas
get_steamspy <- function(app_id, cooldown = 1) {
  url <- paste0("https://steamspy.com/api.php?request=appdetails&appid=", app_id)
  data <- get_response(url)
}
```

```

owners <- get_var(data, c("owners"))

return(owners)
}

api_key <- Sys.getenv("ITAD_KEY")
# Función para consultas a API de ITAD: pico máximo de jugadores
get_itad <- function(app_id) {
  url <- paste0("https://api.isthereanydeal.com/games/lookup/v1?key=", api_key, "&appid=", app_id)
  data <- get_response(url)
  itad_id <- get_var(data, c("game", "id"))

  url <- paste0("https://api.isthereanydeal.com/games/info/v2?key=", api_key, "&id=", itad_id)
  data <- get_response(url)
  players_peak <- get_var(data, c("players", "peak"))

  return(players_peak)
}

# TODO: Pruebas para 5 primeros, esto debe generalizarse (e incorporar tiempos de espera)
# Steam
steam_results <- lapply(df$id[1:5], function(id) get_steam(id))
df[1:5, "total_positive"] <- sapply(steam_results, function(x) x$total_positive)
df[1:5, "total_reviews"] <- sapply(steam_results, function(x) x$total_reviews)
df[1:5, "required_age"] <- sapply(steam_results, function(x) x$required_age)
df[1:5, "pegi"] <- sapply(steam_results, function(x) x$pegi)
# SteamSpy
steamspy_results <- lapply(df$id[1:5], get_steamspy)
df[1:5, "owners"] <- sapply(steamspy_results, function(x) x)
# ITAD
itad_results <- lapply(df$id[1:5], function(id) get_itad(id))
df[1:5, "players_peak"] <- sapply(itad_results, function(x) x)

```

3 Limpieza de los datos

...

```

# Selección de variables relevantes
df <- df %>%
  select(title, date, original_price, achievements_number, langs_number, dlcs_number, has_ost, is_dlc,
         total_positive, total_reviews, required_age, pegi, owners, players_peak)

```

Creación de columnas booleanas para cada género.

```
genre_names <- unique(unlist(strsplit(df$genres, "\\|")))
```

```
genre_cols <- genre_names %>%  
  str_to_lower() %>%  
  str_replace_all("&", "and") %>%  
  str_replace_all("\\s+", "_")  
  
for (i in seq_along(genre_cols)) {  
  col_name <- paste0("is_", genre_cols[i])  
  df[[col_name]] <- str_detect(df$genres, fixed(genre_names[i]))  
}  
  
# TODO: Falta dropear 'genres'
```

4 Análisis de los datos

```
...  
# Guardado del dataset resultante  
write.csv(df, "../datasets/steam_apps_cleaned.csv", row.names = FALSE)
```

5 Representación de los resultados

...

6 Resolución del problema

...

7 Código

...

8 Vídeo

...

Participación de los integrantes

A continuación, se muestra la tabla de firmas que recoge la contribución de cada integrante a la práctica.

Contribuciones	Firma
Investigación previa	<i>Firmar aquí</i>
Redacción de las respuestas	<i>Firmar aquí</i>
Desarrollo del código	<i>Firmar aquí</i>
Participación en el vídeo	<i>Firmar aquí</i>

Referencias

- [1] Subirats, L., Pérez, D.O., Calvo, M. (2019). *Introducción al ciclo de vida de los datos*. [Recurso de aprendizaje textual]. 1.a ed. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).
- [2] Munzert, S., Rubba, C., Meißner, P., Nyhuis, D. (2015). *Automated Data Collection with R: A Practical Guide to Web Scraping and Text Mining*. John Wiley & Sons, Ltd. ISBN: 978-1-118-83481-7